Hosting a Fine-Tuned Qwen Model on a VPS Using vLLM

1. Overview

After fine-tuning `Qwen2.5-7B-Instruct` (or `Qwen3-8B-Instruct`) on the Vedaz astrology conversation data, the merged model needs to be served as a fast, concurrent, OpenAI-API-compatible endpoint. vLLM is used because it gives continuous batching, PagedAttention (efficient KV-cache memory management), and a drop-in `/v1/chat/completions` API — so the existing app backend needs almost no changes to switch from OpenAI's API to the self-hosted model.

Pipeline: train (QLoRA) → merge adapter into base weights → upload merged model to VPS → serve with vLLM → put Nginx + TLS in front → point the app at it.

---

2. Choosing the VPS

Model size	Precision	Min GPU VRAM	Suggested instance
Qwen2.5-1.5B/3B-Instruct	fp16/bf16	8–12 GB	RTX 4000 Ada / A10 (24GB) tier
Qwen2.5-7B-Instruct	fp16/bf16	~16–18 GB	A10G (24GB), L4 (24GB)
Qwen2.5-7B-Instruct	AWQ/GPTQ int4	~6–8 GB	RTX 3060 12GB / A10 shared
Qwen2.5-14B+	fp16	30GB+	A100 40GB

For a chatbot workload like this (short-medium replies, moderate concurrency), a single 24GB GPU VPS (e.g. RunPod, Lambda, Hetzner+GPU, AWS `g5.xlarge`, Vast.ai) serving the 7B model in bf16, or the 7B model quantized to AWQ int4 on a smaller card, is the practical sweet spot. Vedaz's dataset also has a fair amount of Hindi/Hinglish text, so stick with the Instruct tokenizer's default vocab — no changes needed there.

Base OS: Ubuntu 22.04, with an NVIDIA driver + CUDA 12.1+ already installed (most GPU VPS images ship with this).

---

3. Server Setup

```
        # Confirm the GPU is visible
        nvidia-smi

        # System packages
        sudo apt update && sudo apt install -y python3.11 python3.11-venv
git tmux nginx

        # Isolated environment
        python3.11 -m venv ~/vllm-env
        source ~/vllm-env/bin/activate
```

```
# Install vLLM (pulls in a compatible torch build automatically)
pip install --upgrade pip
pip install vllm
```

Check the install:

```
python -c "import vllm; print(vllm.__version__)"
```

---

4. Getting the Model onto the VPS

Copy the **merged** model directory (base weights + LoRA merged in — not the raw adapter) produced by `finetune_qwen_vedaz.py --merge_after`:

```
# from local machine
rsync -avz --progress ./out/qwen2.5-vedaz-lora-merged/ user@vps-ip:/opt/models/qwen-vedaz/
```

Or push the merged model to the Hugging Face Hub as a private repo and pull it down with `huggingface-cli download` — cleaner for repeated deploys/CI.

If VRAM is tight, quantize the merged model to AWQ first (on a machine with enough RAM) using `autoawq`, then serve the quantized version instead.

---

5. Serving with vLLM

Quick manual test:

```
source ~/vllm-env/bin/activate

vllm serve /opt/models/qwen-vedaz \
    --served-model-name vedaz-astrologer \
    --host 0.0.0.0 \
    --port 8000 \
    --dtype bfloat16 \
    --max-model-len 4096 \
    --gpu-memory-utilization 0.90 \
    --api-key "<your-strong-api-key>"
```

Key flags: - `--served-model-name` — the name clients pass in the `model` field. - `--max-model-len` — cap context length to control KV-cache memory; 4096 is comfortable for this astrology-chat use case. - `--gpu-memory-utilization` — fraction of VRAM vLLM is allowed to pre-allocate for KV cache; 0.85–0.90 is a safe default, lower it if you share the GPU. - `--api-key` — enables bearer-token auth on the OpenAI-compatible endpoint. - For quantized weights, add `--quantization awq` (or `gptq`) matching how the model was quantized.

Test it:

```
curl http://localhost:8000/v1/chat/completions \
  -H "Authorization: Bearer <your-strong-api-key>" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "vedaz-astrologer",
    "messages": [
      {"role": "system", "content": "You are Vedaz'\''s AI Vedic astrologer..."},
      {"role": "user", "content": "Mera career kaisa rahega is saal?"}
```

```
            ],
            "temperature": 0.6,
            "max_tokens": 400
          }'
```

---

6. Running It as a Persistent Service (systemd)

Don't leave it running in a `tmux` session in production. Create a systemd unit:

`/etc/systemd/system/vllm-vedaz.service`

```
        [Unit]
        Description=vLLM server - Vedaz astrology model
        After=network.target

        [Service]
        User=ubuntu
        WorkingDirectory=/opt/models
        Environment="PATH=/home/ubuntu/vllm-env/bin"
        ExecStart=/home/ubuntu/vllm-env/bin/vllm serve /opt/models/qwen-
vedaz \
            --served-model-name vedaz-astrologer \
            --host 0.0.0.0 --port 8000 \
            --dtype bfloat16 --max-model-len 4096 \
            --gpu-memory-utilization 0.90 \
            --api-key ${VLLM_API_KEY}
        EnvironmentFile=/etc/vllm-vedaz.env
        Restart=always
        RestartSec=5

        [Install]
        WantedBy=multi-user.target
```

`/etc/vllm-vedaz.env`

```
VLLM_API_KEY=<your-strong-api-key>
```

```
        sudo systemctl daemon-reload
        sudo systemctl enable --now vllm-vedaz
        sudo systemctl status vllm-vedaz
        journalctl -u vllm-vedaz -f     # tail logs
```

---

7. Reverse Proxy + TLS (Nginx + Let's Encrypt)

Don't expose port 8000 directly to the internet. Put Nginx in front with TLS:

`/etc/nginx/sites-available/vedaz-model`

```
server {
    listen 80;
    server_name model.vedaz.example.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_buffering off;          # needed for streaming
responses
```

```
        proxy_read_timeout 300s;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

        sudo ln -s /etc/nginx/sites-available/vedaz-model /etc/nginx/sites-
enabled/
        sudo nginx -t && sudo systemctl reload nginx

        # TLS certificate
        sudo apt install -y certbot python3-certbot-nginx
        sudo certbot --nginx -d model.vedaz.example.com
```

Also lock down the firewall so only 80/443/22 are open, and port 8000 is only reachable via `localhost`:

```
        sudo ufw allow OpenSSH
        sudo ufw allow 'Nginx Full'
        sudo ufw enable
```

---

8. Client Integration

Because vLLM speaks the OpenAI API schema, the app backend just needs a new base URL and key — no SDK changes:

```
        from openai import OpenAI

        client = OpenAI(
            base_url="https://model.vedaz.example.com/v1",
            api_key="<your-strong-api-key>",
        )

        resp = client.chat.completions.create(
            model="vedaz-astrologer",
            messages=[
                {"role": "system", "content": "You are Vedaz's AI Vedic
astrologer..."},
                {"role": "user", "content": "Mera career kaisa rahega is
saal?"},
            ],
            temperature=0.6,
            max_tokens=400,
        )
        print(resp.choices[0].message.content)
```

---

9. Monitoring & Operational Notes

- **GPU/queue health:** `nvidia-smi -l 2` while under load; vLLM also exposes Prometheus metrics at `/metrics` (add `--otlp-traces-endpoint`/`--metrics` flags as needed) — wire into Grafana if you want dashboards.
- **Throughput tuning:** raise `--max-num-seqs` for more concurrent requests if VRAM allows; vLLM's continuous batching handles the scheduling.
- **Zero-downtime updates:** deploy the new merged model to a fresh directory, bring up a second vLLM instance on a different port, smoke-test it, then flip the Nginx `proxy_pass` and reload — avoids killing in-flight requests.
- **Cost control:** GPU VPS billing is usually hourly — if traffic is bursty, consider a scale-to-zero setup (RunPod Serverless, Modal, etc.) instead of a systemd-always-on box; the vLLM serving

command stays the same, only the orchestration layer changes.
- **Safety layer:** since this model is meant to stay within Vedaz's non-fatalistic, no-medical/legal-diagnosis guidelines, keep the system prompt fixed server-side (not client-editable) and consider a lightweight moderation/guardrail check on outputs before they reach the user, since fine-tuning tightens typical behavior but doesn't guarantee it under adversarial prompting.

---

10. Quick Reference — Full Command Sequence

```
        # 1. Environment
        python3.11 -m venv ~/vllm-env && source ~/vllm-env/bin/activate
        pip install vllm

        # 2. Get model onto VPS
        rsync -avz ./out/qwen2.5-vedaz-lora-merged/
user@vps:/opt/models/qwen-vedaz/

        # 3. Serve
        vllm serve /opt/models/qwen-vedaz \
            --served-model-name vedaz-astrologer \
            --host 0.0.0.0 --port 8000 \
            --dtype bfloat16 --max-model-len 4096 \
            --gpu-memory-utilization 0.90 \
            --api-key "$VLLM_API_KEY"

        # 4. Put behind systemd + Nginx + TLS (see sections 6-7)
```